

Systems Analysis and Design

fliptop.3d: A Graph-Based Visualization of Battle
Rap Networks

Kristian Mark Surat

201502678

B Library and Information Science
LIS THY: Data Structures for LIS
Second Semester, Academic Year 2025-2026
UP SLIS, UP Diliman

Submitted to:
Asst. Professor Janny Surmieda

June 5, 2026

Abstract

This document outlines the Systems Analysis and Design (SAD) for the flip-top.3d project. The system is designed to provide a graph-based visualization of battle rap networks within the Fliptop Battle League in the Philippines. It models Emcees as nodes and their battle outcomes as directed or undirected edges. The architecture relies on a Command Query Responsibility Segregation (CQRS) pattern, utilizing PostgreSQL for relational data storage, Neo4j for complex graph traversal and rendering, and Upstash Redis for high-performance caching. This SAD provides a comprehensive view of the system's architecture, data models, diagrams, and underlying technologies.

Contents

1	Introduction	3
1.1	Background and Context	3
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Availability & Open Source	3
2	System Requirements	4
2.1	Functional Requirements	4
2.2	Non-Functional Requirements	4
3	System Architecture	4
3.1	Deployment Architecture	5
4	System Models	5
4.1	Database Schema Diagrams	5
4.1.1	Relational Model (Supabase/PostgreSQL)	5
4.1.2	Property Graph Model (Neo4j)	6
4.2	Unified Modeling Language (UML) Diagrams	7
4.2.1	Use Case Diagram	7
4.2.2	Class Diagram	8
4.2.3	Sequence Diagram	9
4.3	Data Flow Diagram (DFD)	10
5	Database Design	11
5.1	Relational Model (Supabase/PostgreSQL)	11
5.2	Graph Model (Neo4j)	12
6	Subsystem and Interface Design	12
6.1	Interactive 3D Graph Rendering Module	12
6.2	Administrative Synchronization Module	12
7	User Interface & Experience (UI/UX)	13
7.1	Visual Legend & Symbology	13
7.2	System Functionalities	13
7.3	Interactive Controls & Navigation	14
7.4	Core Visualization Interface	14
7.5	Interactive Overlays	16
7.6	Mobile Responsiveness & Interface	17
7.7	Administrative Dashboard	18
8	Technology Stack	20
8.1	Hardware & Software Specifications	20
9	System Limitations	20
10	Recommendations & Future Enhancements	20
11	Conclusion	21

1 Introduction

1.1 Background and Context

Founded in 2010 by Alaric Riam Yuson (Anygma), the FlipTop Battle League started as a small, passionate underground movement. Since its first event, "Grain Assault", it has completely exploded into the most-viewed professional rap battle conference in the world. But FlipTop is more than just entertainment—it's essentially the modern-day "Balagtasan". It breathed life back into Philippine hip-hop, giving artists a raw, uncensored platform to showcase their lyrical mastery, bilingual wordplay, and sharp social commentary [8]. Over the past decade, FlipTop has built a massive, breathing ecosystem of regional divisions, unforgettable matchups, deep-seated rivalries, and a constantly shifting hierarchy of Emcees.

1.2 Problem Statement

Despite FlipTop's massive cultural footprint and rich history spanning well over a decade, keeping track of its ever-growing battle network is incredibly tough. When you try to look at all these battles, regional allegiances, and rivalry chains through simple spreadsheets or chronological video feeds, the magic is lost. Traditional databases completely hide the true connections between emcees—you can't see who the real central figures are, how veterans connect to newcomers, or the complex pathways that define the league's hierarchy. Because of this, fans, cultural researchers, and even the organizers are left without a unified, interactive map that truly brings the macroscopic structure of the FlipTop universe to life.

1.3 Objectives

This project aims to solve the visualization problem by leveraging graph data structures. By modeling individual Emcees as vertices (nodes) and their battle outcomes as directed or undirected edges, the platform provides a novel, interactive methodology for:

- Observing league hierarchies.
- Measuring network centrality (e.g., distinguishing veteran participants from newcomers).
- Visualizing historical battle paths interactively in a 3D space.

1.4 Availability & Open Source

As of June 5, 2026, the compiled application is deployed and publicly accessible online at <https://fliptop3d.shansurat.dev>. The complete source code and technical documentation are available on GitHub at <https://github.com/shansurat/fliptop.3d>.

2 System Requirements

2.1 Functional Requirements

- **Graph Traversal & Visualization:** The system must parse complex node-edge data and render an interactive 3D WebGL physics-based graph.
- **Dynamic Filtering:** The system must allow users to filter graph data temporally (by year) and structurally (node scaling by views vs. battles).
- **Administrative Data Entry:** The system must provide a secure gateway for authorized administrators to perform CRUD (Create, Read, Update, Delete) operations on relational data.
- **Data Synchronization:** The system must allow administrators to safely synchronize relational records into the graph database.

2.2 Non-Functional Requirements

- **Performance:** Initial graph rendering must leverage edge-caching to ensure sub-100ms load times, avoiding direct heavy queries against the graph database on every client request.
- **Security:** Administrative subsystems must be protected via robust authentication (e.g., JWT-based auth mechanisms), preventing unauthorized data mutation.
- **Scalability:** The read-heavy architecture must be fully decoupled from the write-heavy architecture to ensure high availability during traffic spikes.

3 System Architecture

The system architecture utilizes a Command Query Responsibility Segregation (CQRS) inspired pattern to separate relational data storage (writes) from graph-based read models (reads).

1. **Relational Source of Truth (Supabase/PostgreSQL):** Raw data pertaining to Emcee profiles and Battle outcomes are initially stored in normalized relational tables. This ensures data integrity, enforces strict schemas, and simplifies administrative data entry.
2. **Graph Read Model (Neo4j):** Data is synchronized into a Neo4j graph database. This specialized database excels at modeling interconnected entities and allows for highly efficient pathfinding and sub-graph highlighting.
3. **Caching Layer (Upstash Redis):** To optimize performance, the server executes complex Cypher traversal queries against Neo4j and caches the resulting JSON payloads in Redis. This facilitates near-instantaneous 3D client rendering without overloading the Neo4j instance.

3.1 Deployment Architecture

The system leverages a modern cloud-native, serverless architecture to ensure high availability and global edge distribution:

- **Frontend & API Gateway:** Deployed on **Vercel**, utilizing Edge Functions and Server-Side Rendering (SSR) for optimal performance.
- **Relational Database Host:** Hosted on **Supabase Cloud**, providing a managed PostgreSQL instance and secure authentication services.
- **Graph Database Host:** Hosted on **Neo4j AuraDB**, a fully managed cloud graph database service optimized for complex Cypher queries.
- **Caching Host:** Hosted on **Upstash Redis**, providing serverless, globally distributed in-memory caching.

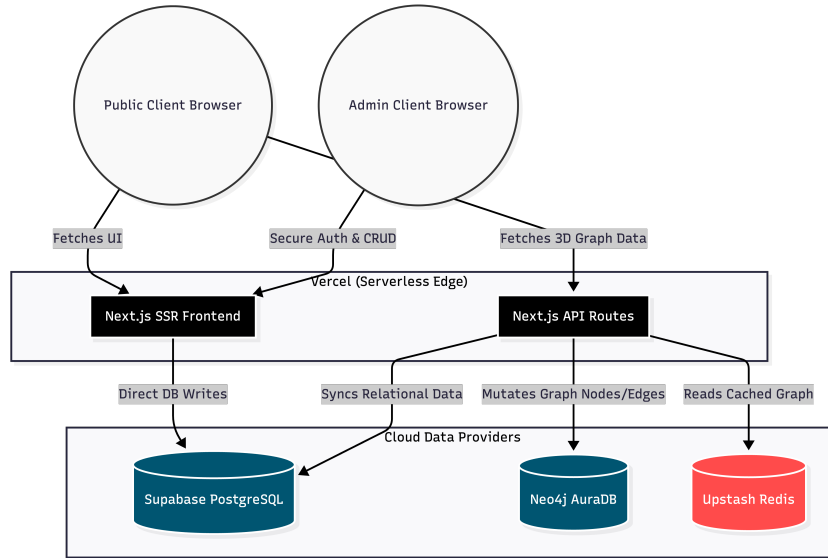


Figure 1: Deployment Architecture Diagram

4 System Models

This section outlines the visual models used to understand and design the system's behavior and structure.

4.1 Database Schema Diagrams

This section visually outlines the structures of our two disparate data sources.

4.1.1 Relational Model (Supabase/PostgreSQL)

The Relational ERD focuses strictly on the core entities that support the graph model (emcees, battles, and battle_participants).

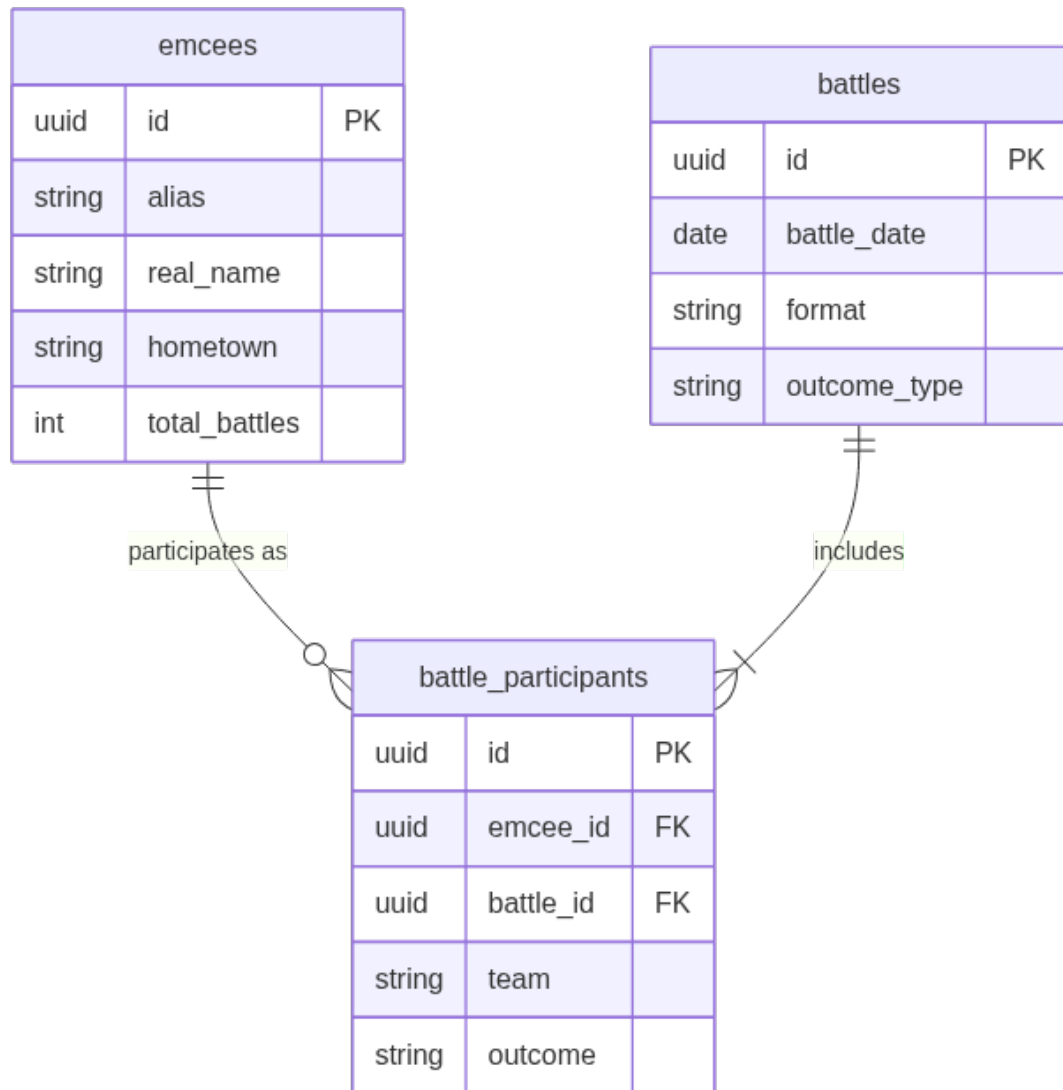


Figure 2: Relational Entity-Relationship Diagram (Supabase)

4.1.2 Property Graph Model (Neo4j)

The Neo4j schema visualizes how the structured relational data is transformed into a node-edge network for traversal.

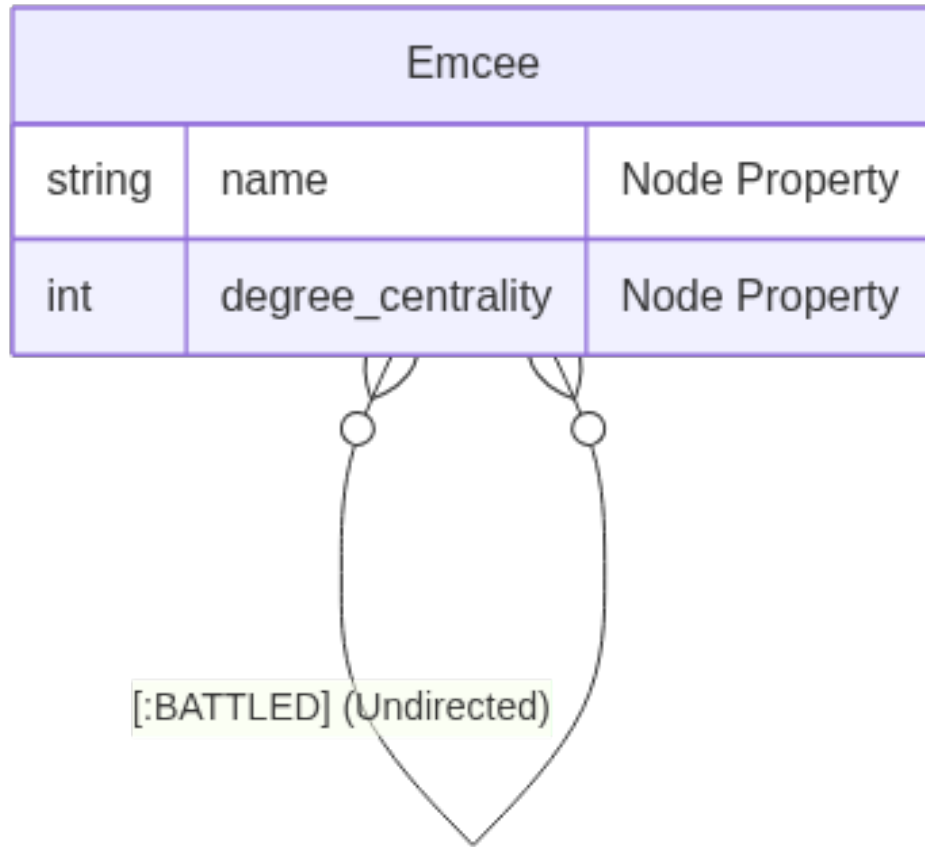


Figure 3: Property Graph Model Schema (Neo4j)

4.2 Unified Modeling Language (UML) Diagrams

UML diagrams describe the behavioral and structural aspects of the flitop.3d system.

4.2.1 Use Case Diagram

The Use Case Diagram defines the interactions between the system's actors (e.g., Public User, Administrator) and the system itself.

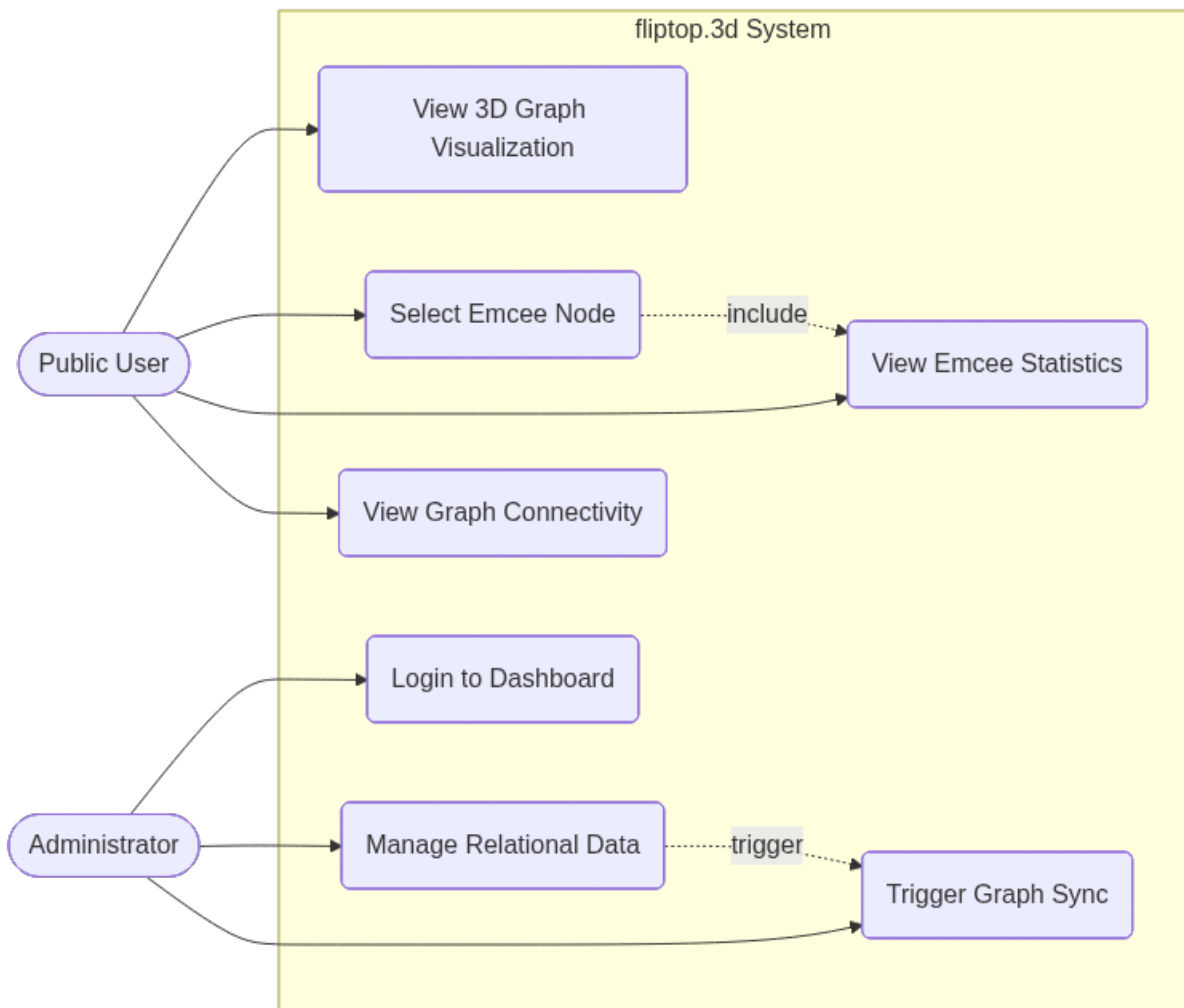


Figure 4: Use Case Diagram

4.2.2 Class Diagram

The Class Diagram illustrates the object-oriented structure of the backend services and frontend components, particularly how the Next.js API routes interact with the Supabase client, Neo4j driver, and Upstash Redis.

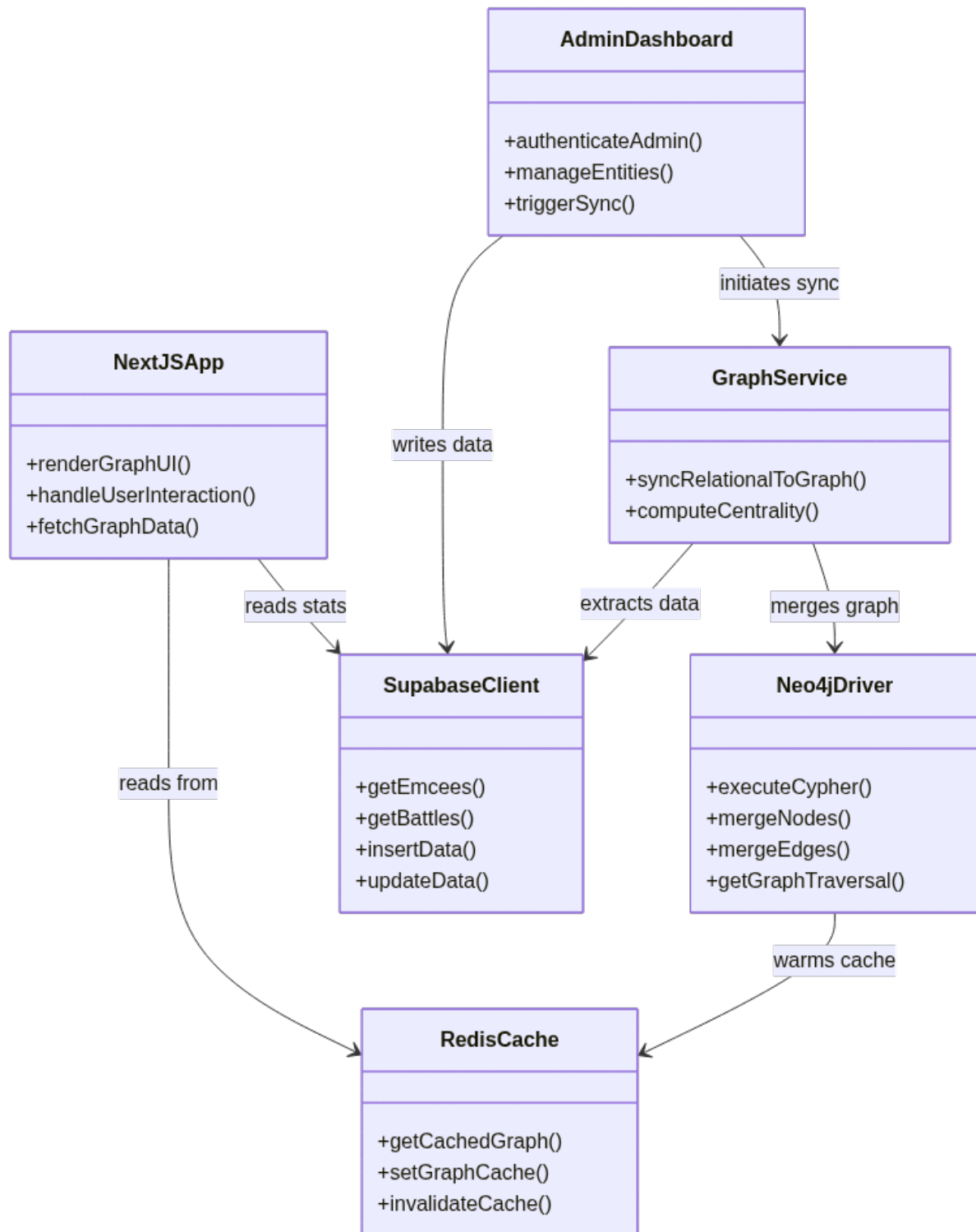


Figure 5: Class Diagram

4.2.3 Sequence Diagram

The Sequence Diagram outlines the dynamic behavior and specific API calls involved when an Administrator synchronizes relational data (Supabase) to the graph model (Neo4j) and caches the result for the public client (Redis).

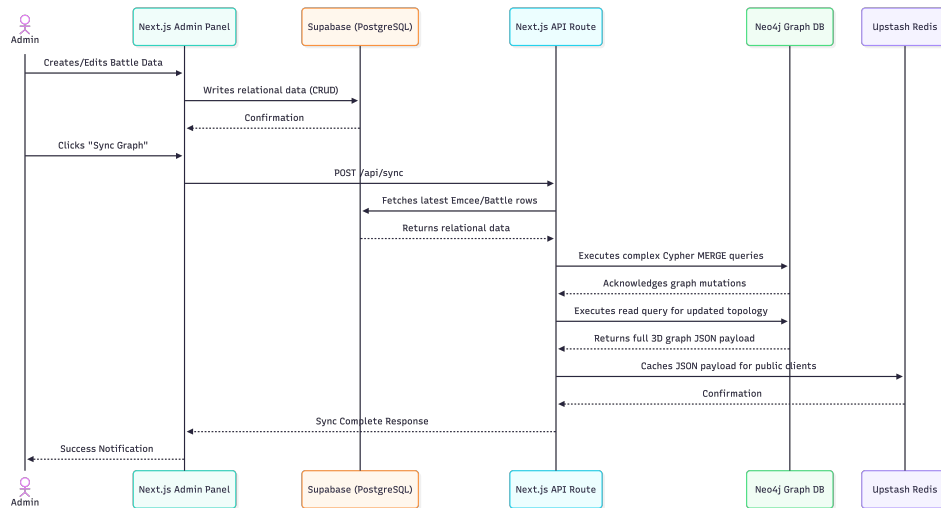


Figure 6: Sequence Diagram (Data Synchronization Flow)

4.3 Data Flow Diagram (DFD)

The Data Flow Diagram tracks the movement of information through the system. Specifically, it highlights the CQRS pipeline: how an admin submits new battle data, how it is written to Supabase, how the synchronization routine pushes it to Neo4j, and how the client reads the cached graph data from Redis.

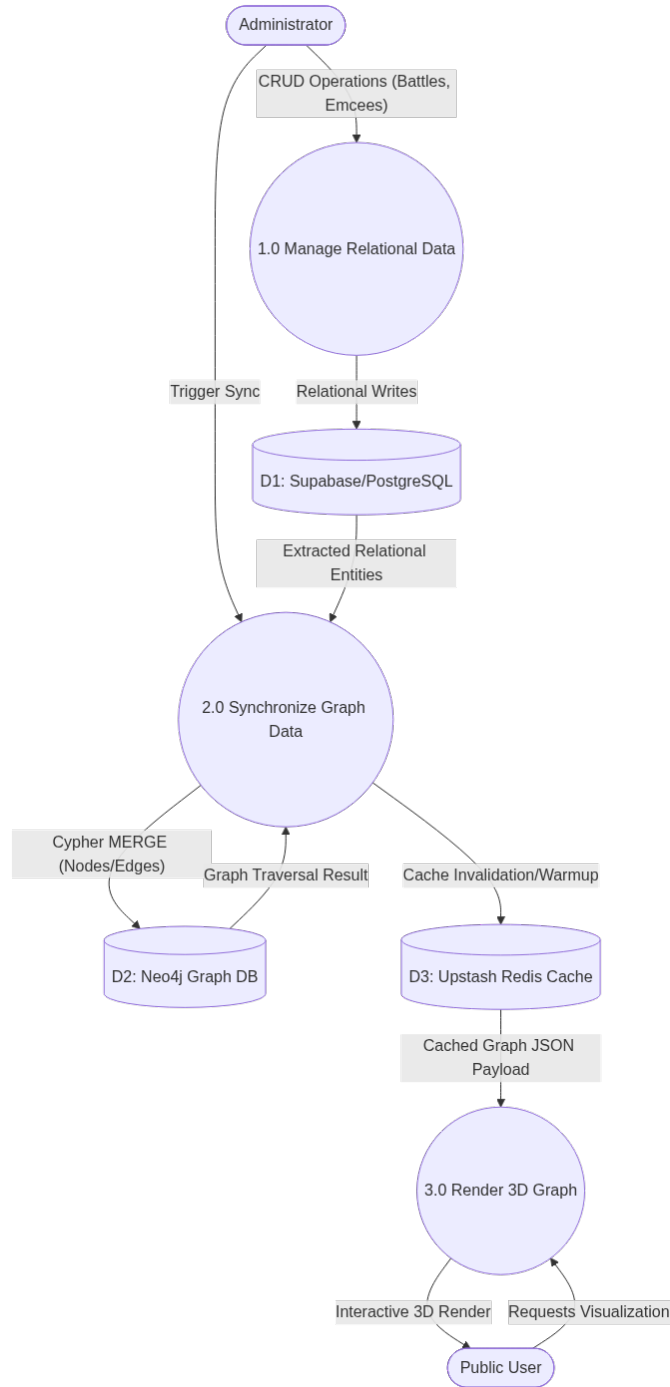


Figure 7: Data Flow Diagram (Level 0/1)

5 Database Design

5.1 Relational Model (Supabase/PostgreSQL)

The relational database acts as the primary source of truth, structured in Third Normal Form (3NF) to ensure data integrity and prevent data anomalies. While the database contains numerous entities for media, events, and judging, the core schema that explicitly drives the graph visualization revolves around the following primary entities:

- **Emcees (emcees):** The central entity representing individual battle rappers. It stores the unique identifier, alias, real name, and biographical metadata.
- **Battles (battles):** Stores individual matchup details. It tracks metadata like battle format and general outcomes.
- **Battle Participants (battle_participants):** A crucial junction table resolving the many-to-many relationship between **emcees** and **battles**. It explicitly defines the role of the emcee in the battle and records their specific side or outcome in the matchup.

This normalized structure provides a highly structured, queryable foundation for the administrative backend, ensuring that the graph synchronization routines can seamlessly extract the **emcees** and their **battle_participants** connections to generate the graph's nodes and edges.

5.2 Graph Model (Neo4j)

The graph database is highly optimized for read-heavy operations and UI rendering. The schema is strictly defined:

- **Nodes:** Represent **Emcee** entities. Properties include name and degree (for centrality visualization).
- **Edges:** Represent relationships.
 - A **DEFEATED** edge is directed from the victor to the defeated.
 - A **BATTLED** edge is bidirectional, representing a draw or an unjudged promotional match.

6 Subsystem and Interface Design

6.1 Interactive 3D Graph Rendering Module

The primary user interface relies on WebGL mapping parsed graph data into a spatial physics-based layout.

- **Degree Centrality Visualization:** The rendering algorithm dynamically scales node sizes based on an Emcee's total number of edges (battle count). This instantly visually distinguishes veteran participants from newcomers.
- **Node-Specific Subgraph Highlighting:** Selecting a node triggers a UI overlay containing the Emcee's aggregated statistics and dims all unrelated edges in the 3D space. This interaction emphasizes the selected entity's direct battle network.

6.2 Administrative Synchronization Module

A secure dashboard is provided for data management. It allows administrators to perform CRUD operations directly on the relational database. Subsequently, users can trigger synchronization routines that map the relational data into graph structures using complex Cypher **MERGE** operations to ensure idempotency.

7 User Interface & Experience (UI/UX)

The system’s UI is designed with a dark-mode aesthetic to emphasize the vibrant colors of the 3D graph nodes and edges. The interface is intentionally minimalist to prevent cognitive overload, allowing the spatial data to be the focal point of the user experience.

7.1 Visual Legend & Symbolology

To effectively communicate the complex data within the 3D space, the system employs a strict visual legend:

- **Node Colors (Win Rate):** An Emcee’s node color operates on an HSL spectrum mapping their career Win Rate. A 0% win rate renders as **Red**, a 50% win rate as **Yellow**, and a 100% win rate as **Green**.
- **Edge Colors (Match Types):** Links between nodes are color-coded by the event format:
 - **Metallic Gold:** Official Tournament Matches.
 - **Slate Grey:** Standard Judged Battles.
 - **Magenta:** Promotional Matches.
 - **Blue:** Tryout Matches.
- **Directional Particles:** Animated light particles travel along the edges to represent outcomes. In standard unselected views, **Gold** particles continuously flow along Tournament edges. When a specific Emcee node is selected, particles flow rapidly along their connecting edges: **Green particles** flowing inward represent a victory for the selected Emcee, while **Red particles** flowing outward represent a defeat.

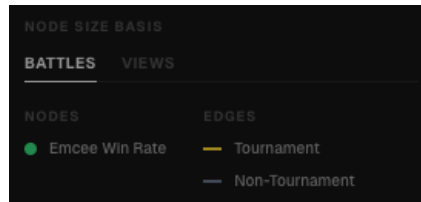


Figure 8: On-Screen Visual Legend (Browser: Chrome 149, OS: Mac OS Sonoma)

7.2 System Functionalities

The application features several dynamic interactive capabilities for exploring the graph:

- **Dynamic Node Sizing:** Node sizes can be toggled to scale by the total number of *battles* an emcee has participated in, or by their total accumulated *views*.
- **Temporal Filtering:** Users can filter nodes by specific years. By default, the system displays *All Years*, rendering all nodes alongside both tournament and non-tournament links.

7.3 Interactive Controls & Navigation

The floating control menu allows rapid manipulation of the graph's rendering parameters.

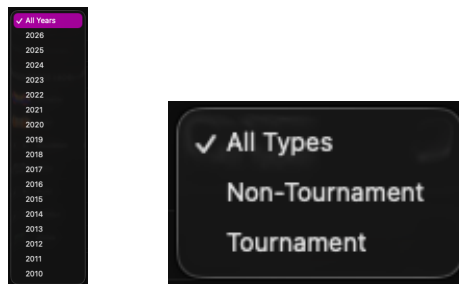


Figure 9: Temporal and Match Type Filters (Browser: Chrome 149, OS: Mac OS Sonoma)

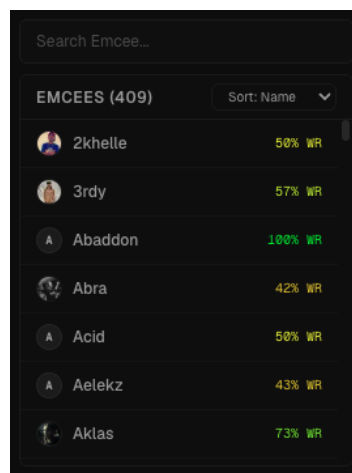


Figure 10: Emcee Search and Sort Panel (Browser: Chrome 149, OS: Mac OS Sonoma)

7.4 Core Visualization Interface

The core screen occupies the full viewport, rendering the WebGL canvas with interactive graph filters and overlay panels.



Figure 11: Default Home View showing all years and link types (Browser: Chrome 149, OS: Mac OS Sonoma)

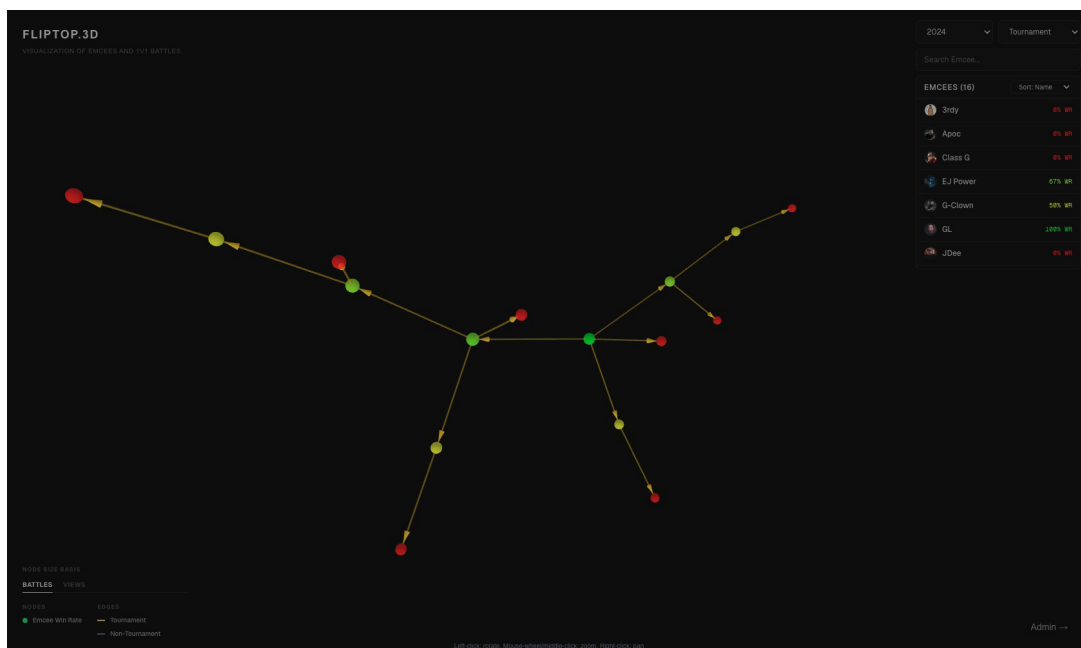


Figure 12: Graph filtered by Year 2024 with nodes sized by Total Battles (Browser: Chrome 149, OS: Mac OS Sonoma)

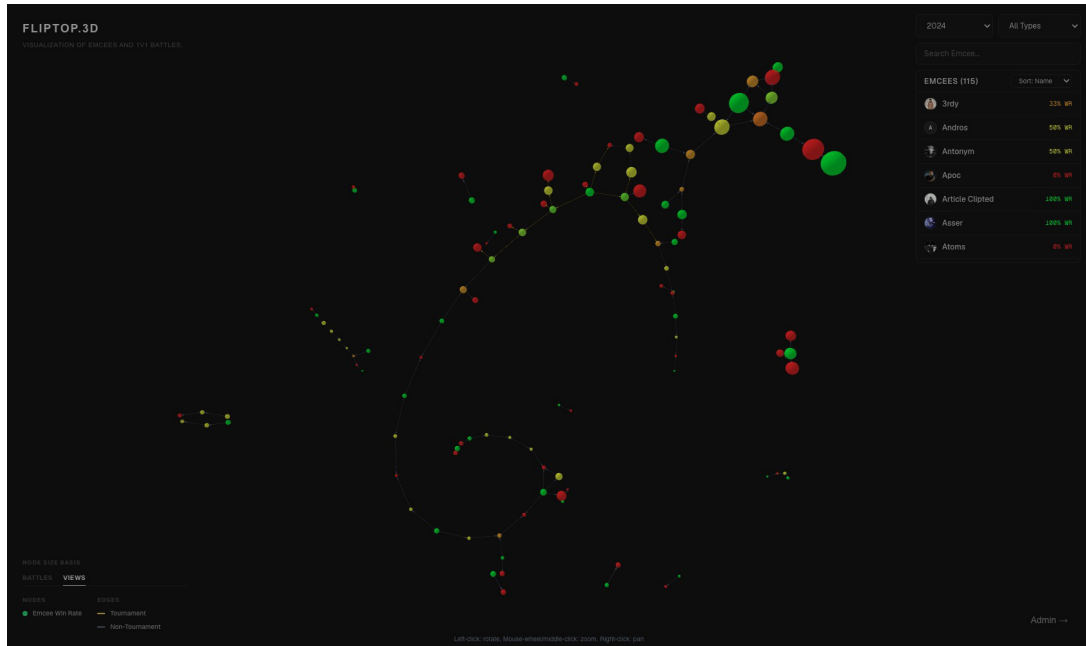


Figure 13: Graph filtered by Year 2024 with nodes sized by Total Views (Browser: Chrome 149, OS: Mac OS Sonoma)

7.5 Interactive Overlays

Selecting specific entities triggers a UI overlay containing aggregated statistics while keeping the graph context visible.



Figure 14: Emcee Node Selection Overlay (Browser: Chrome 149, OS: Mac OS Sonoma)

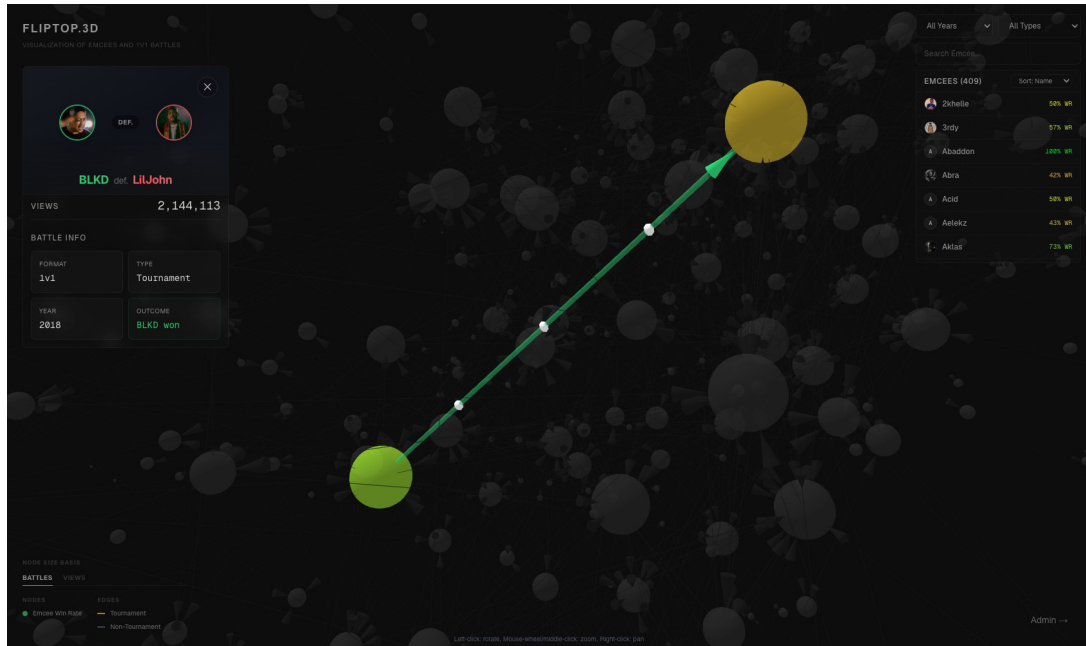


Figure 15: Battle Selection Overlay (Browser: Chrome 149, OS: Mac OS Sonoma)

7.6 Mobile Responsiveness & Interface

The application is fully responsive, ensuring the 3D graph visualization and core interactions remain accessible on mobile devices. Note that administrative capabilities are restricted to desktop environments; there is no admin dashboard access provided on mobile devices to ensure data integrity and prevent accidental CRUD operations on smaller touch screens.

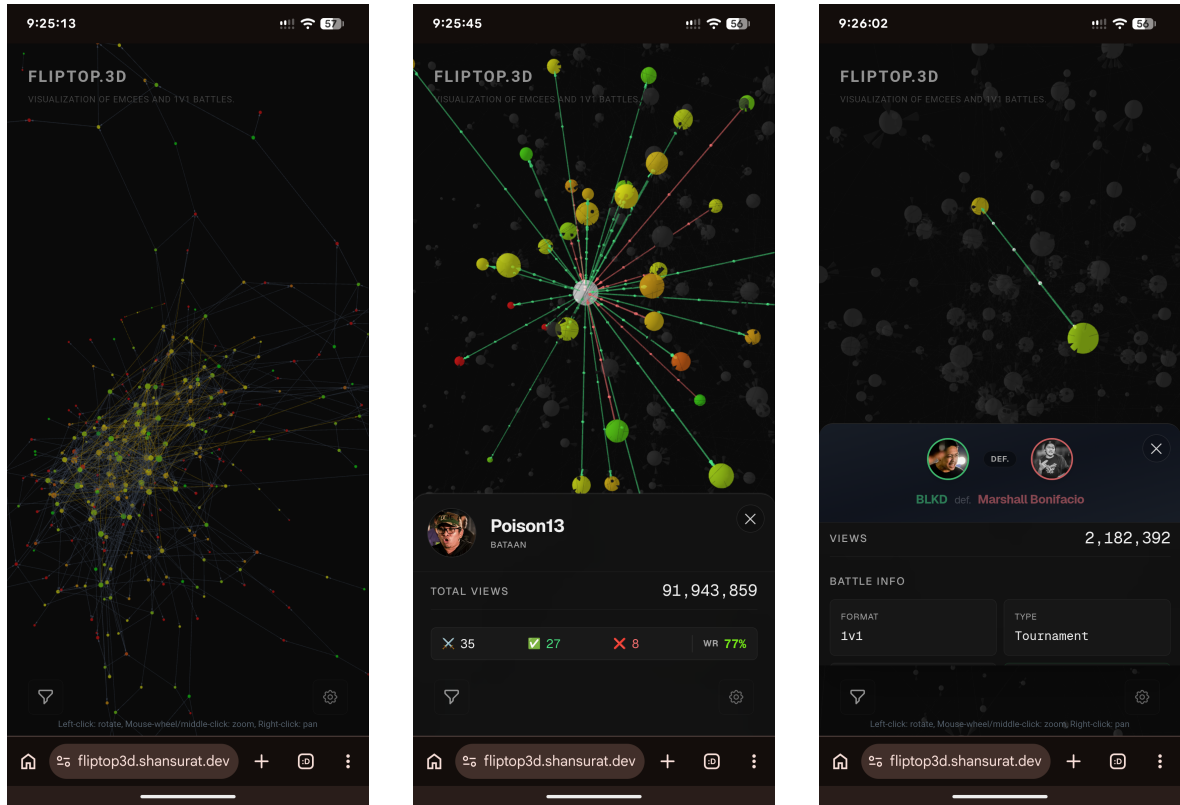


Figure 16: Mobile Interface Views (Browser: Chrome 149, OS: Android, Device: Pixel 7A)

7.7 Administrative Dashboard

The secure dashboard utilizes traditional tabular layouts optimized for CRUD operations, providing administrators with high-density data views for rapid editing. Access to this subsystem is heavily gated by an authentication layer and is strictly restricted to desktop environments (mobile access is disabled).

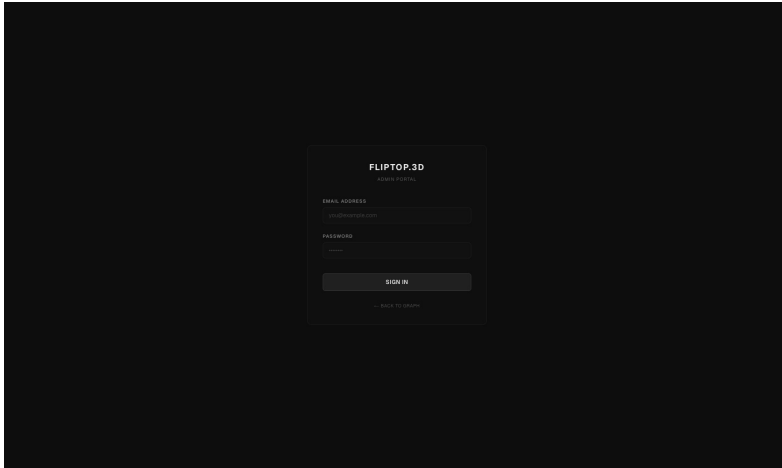


Figure 17: Administrative Secure Login Gateway (Browser: Chrome 149, OS: Mac OS Sonoma)

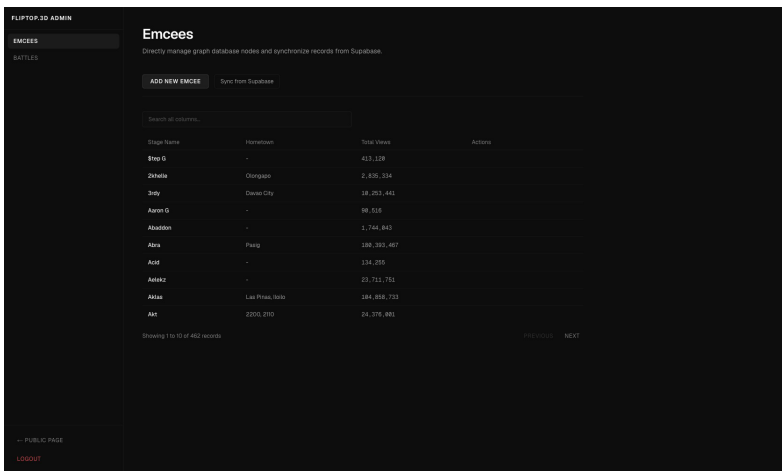


Figure 18: Admin Dashboard for Emcees (Browser: Chrome 149, OS: Mac OS Sonoma)

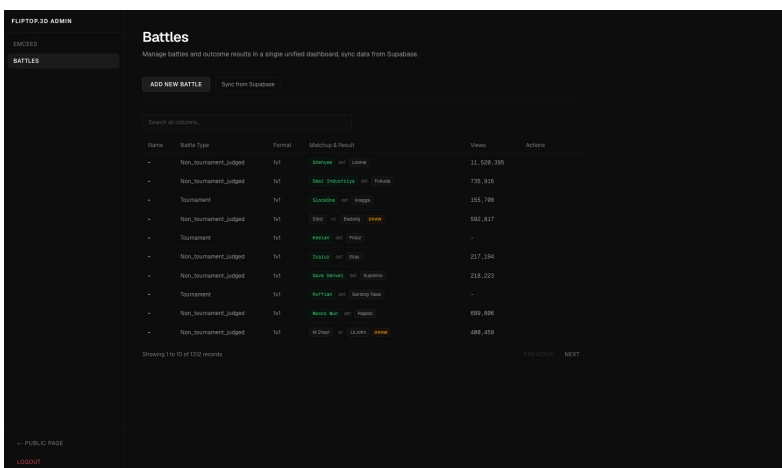


Figure 19: Admin Dashboard for Battles (Browser: Chrome 149, OS: Mac OS Sonoma)

8 Technology Stack

The system is built using modern, highly scalable web technologies:

- **Frontend Environment:** Next.js 16 (App Router), React 19, Tailwind CSS v4.
- **Visualization Library:** `react-force-graph-3d` layered over Three.js.
- **Relational Database:** Supabase (PostgreSQL).
- **Graph Database:** Neo4j.
- **Caching Mechanism:** Upstash Redis.

8.1 Hardware & Software Specifications

Due to the computationally intensive nature of mapping large datasets into physics-based 3D layouts, specific operational environments are required:

- **Client Requirements:** A modern web browser (e.g., Chrome, Safari, Edge) running on a device with hardware-accelerated WebGL/GPU support to ensure smooth 60fps graph rendering.
- **Server Environment:** A Node.js runtime environment capable of supporting Next.js 16 serverless execution contexts.

9 System Limitations

While the graph database provides a comprehensive overview of the battle rap network, it is important to note that the system accurately maps approximately 91% of the overall results and battle data. The remaining percentage accounts for undocumented, unjudged, or lost historical footage that cannot be definitively verified.

10 Recommendations & Future Enhancements

To further maximize the analytical and historical value of the `fliptop.3d` graph, several architectural and data modeling enhancements are highly recommended for future iterations:

- **Expanded Entity Modeling (Judges & Crews):**
 - *Judging Bias Analysis:* Introducing "Judge" nodes connected via *VOTED_FOR* edges would allow the graph to visualize judging patterns, revealing whether certain judges consistently favor specific styles or specific emcees.
 - *Faction Rivalries:* Implementing "Crew" or "Clan" hyper-nodes (e.g., Uprising, Dongalo Wreckords) would enable visualizations of intra-crew alliances and large-scale inter-crew rivalries across the network's history.
- **Data Enrichment & Advanced Analytics:**

- *Algorithmic Rankings*: Calculating and assigning Win-Loss Elo ratings or mathematical "Impact Scores" (PageRank) to nodes would provide a quantitative topological dominance metric, rather than relying solely on raw view counts.
 - *Metric Diversification*: Including crowd voting statistics versus official judging panel results to highlight controversial match outcomes on the graph.
- **Advanced Temporal Visualizations:**
 - *Chronological Playback*: Developing a dynamic animation timeline that visually "grows" the 3D network year-over-year, allowing users to actively observe the physical expansion of the battle rap scene from its inception to the present day.
 - **Crowdsourcing Missing Data:**
 - *Community Verification Portal*: To address the current 9% gap in historical data (due to lost or unjudged footage), the system should implement a secure, authenticated community-driven portal where super-fans can submit metadata and historical proofs to help complete the graph.

11 Conclusion

The fliptop.3d system provides a highly interactive and insightful method for understanding the dense interconnected history of the Philippine battle rap scene. By strategically splitting data concerns via CQRS—utilizing PostgreSQL for administrative data integrity, Neo4j for graph relationships, and Redis for edge-cached rendering—the platform is built to be both robust and highly performant.

References

- [1] Vercel Inc. (2026). *Next.js: The React Framework for the Web*. Retrieved from <https://nextjs.org/>
- [2] Supabase Inc. (2026). *Supabase: The Open Source Firebase Alternative*. Retrieved from <https://supabase.com/>
- [3] Neo4j, Inc. (2026). *Neo4j Graph Database & Analytics*. Retrieved from <https://neo4j.com/>
- [4] Upstash, Inc. (2026). *Upstash: Serverless Data for Redis*. Retrieved from <https://upstash.com/>
- [5] Cabello, R., et al. (2026). *Three.js: JavaScript 3D Library*. Retrieved from <https://threejs.org/>
- [6] Asturiano, V. (2026). *react-force-graph: React bindings for 2D/3D force-directed graphs*. Retrieved from <https://github.com/vasturiano/react-force-graph>

- [7] Fowler, M. (2011). *CQRS (Command Query Responsibility Segregation)*. Retrieved from <https://martinfowler.com/bliki/CQRS.html>
- [8] FlipTop Battle League. (2026). *Official FlipTop Website*. Retrieved from <https://www.fliptop.com.ph/>